

# **Layer 4: Data & Oracle Layer**

Vertical Decomposition for Agentic IDE Implementation

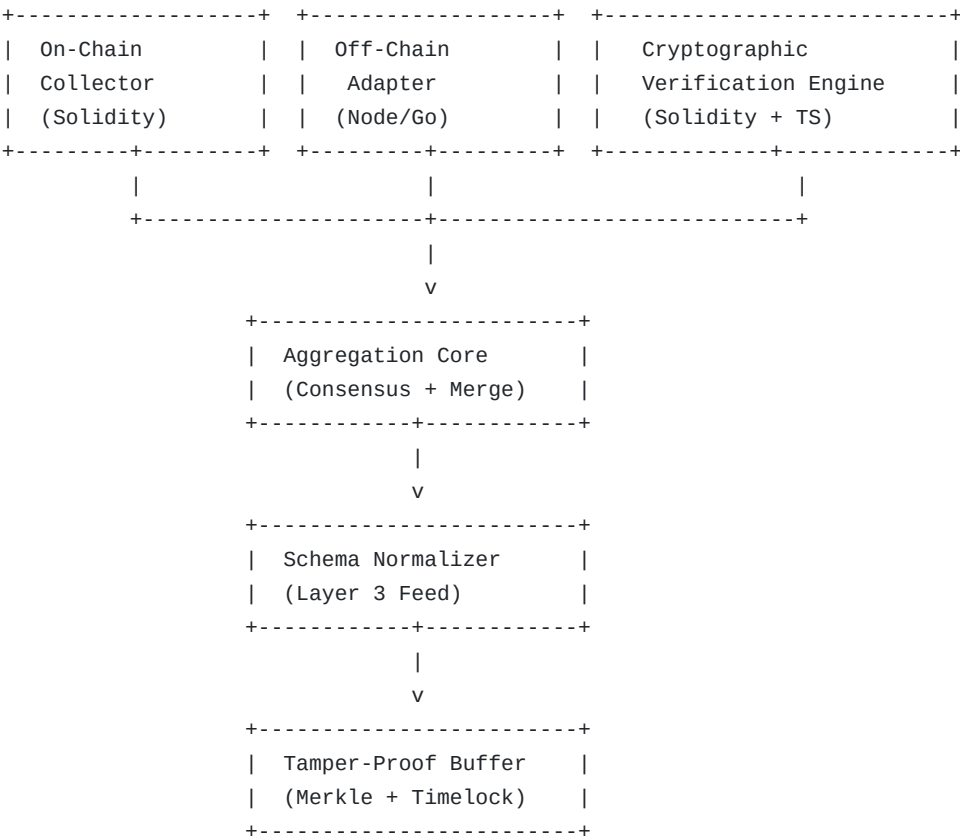
DAO Governance Architecture - Self-Evolving Quadratic Voting Module

*Generated: 2026-05-06*

# 1. Architecture Overview

Layer 4 serves as the sensory input layer of the DAO Governance Architecture. Its sole purpose is to feed verified, tamper-proof data into the Intelligence Layer (Layer 3) so it can accurately calculate optimization parameters.

## High-Level Component Map



The layer is designed as a pipeline: raw data enters from multiple sources, is cryptographically verified, aggregated through consensus, normalized to a strict schema, and buffered through a tamper-proof commit-reveal mechanism before Layer 3 consumes it.

## 2. Sub-Module Decomposition (Agent-Ready)

### Module 4A: On-Chain Metrics Collector

Responsibility: Extract raw on-chain state for voter turnout, treasury velocity, token metrics, and voice credit utilization.

#### File Inventory & Agent Tasks

| File                                     | Purpose                                         | Agent Task                                                             |
|------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------|
| contracts/l4/OnChainDataProvider.sol     | Read-only view contract aggregating DAO state   | Implement getVoterTurnout(), getTreasuryVelocity(), getTokenPrice() vi |
| contracts/l4/interfaces/IOnChainData.sol | Struct definitions for on-chain metrics         | Define OnChainMetrics struct with exact types                          |
| scripts/l4/collect-onchain.ts            | Hardhat/Foundry script to batch-collect metrics | Script that calls view functions and emits to Aggregation Core         |

#### Key Interface (Solidity)

```
interface IOnChainData {
    struct OnChainMetrics {
        uint256 proposalId;
        uint256 voterTurnoutBps;           // Basis points (0-10000)
        uint256 totalVoiceCreditsUsed;
        uint256 treasuryInflow;
        uint256 treasuryOutflow;
        uint256 tokenPriceUsd;           // 8 decimal places
        uint256 timestamp;
        bytes32 blockHash;
    }

    function getMetrics(uint256 proposalId)
        external view returns (OnChainMetrics memory);
}
```

## Module 4B: Off-Chain Data Adapter

Responsibility: Ingest quantifiable off-chain data including market prices, sentiment signals, macro indicators, and identity verification status.

### File Inventory & Agent Tasks

| File                                           | Purpose                                     | Agent Task                                    |
|------------------------------------------------|---------------------------------------------|-----------------------------------------------|
| services/l4/adapters/BaseAdapter.ts            | Abstract class for all off-chain adapters   | Define fetch(), verify(), normalize() methods |
| services/l4/adapters/ChainlinkMarketAdapter.ts | Token price, volatility via Chainlink feeds | Integrate Chainlink AggregatorV3Interface     |
| services/l4/adapters/SentimentAdapter.ts       | Social/sentiment signals                    | REST API wrapper with HMAC auth               |
| services/l4/adapters/ProofOfHumanityAdapter.ts | Identity verification status checks         | PoH subgraph or contract queries              |
| services/l4/types/OffChainData.ts              | Union types for all off-chain payloads      | Type-safe schema definitions                  |

### Key Interface (TypeScript)

```
interface OffChainDataPacket {
  source: string;           // e.g., "chainlink-eth-usd"
  dataType: DataType;      // ENUM: PRICE | SENTIMENT | IDENTITY
  payload: unknown;
  timestamp: number;
  signature: string;        // ECDSA from source oracle node
  rawUrl: string;          // For audit trail
}
```

## Module 4C: Cryptographic Verification Engine

Responsibility: Ensure every data input is cryptographically verifiable before entering the aggregation pipeline. No unverified data may proceed.

### File Inventory & Agent Tasks

| File                                    | Purpose                                          | Agent Task                                             |
|-----------------------------------------|--------------------------------------------------|--------------------------------------------------------|
| contracts/l4/VerificationRegistry.sol   | On-chain registry of approved oracle public keys | Atomic managed key registry                            |
| services/l4/crypto/SignatureVerifier.ts | Off-chain ECDSA + EIP-712 signature validation   | Verify oracle signatures against registry              |
| services/l4/crypto/MerkleBuilder.ts     | Build Merkle tree of aggregated data batches     | Generate proofs for Layer 3 consumption                |
| contracts/l4/DataVerifier.sol           | On-chain Merkle proof verification               | verifyBatch(bytes32 root, bytes32[] proof, bytes leaf) |

### Verification Flow

Raw Data -> Signature Check (ECDSA) -> Source Registry Check -> Merkle Leaf Hash -> Aggregation

## Module 4D: Aggregation Core (Consensus + Merge)

Responsibility: Multi-source aggregation with outlier detection and manipulation resistance. This is the mathematical heart of Layer 4.

### File Inventory & Agent Tasks

| File                                        | Purpose                                       | Agent Task                           |
|---------------------------------------------|-----------------------------------------------|--------------------------------------|
| services/l4/aggregation/Aggregator.ts       | Main orchestrator pulling from all adapters   | Implement polling/cron logic         |
| services/l4/aggregation/ConsensusEngine.ts  | Median/mean consensus with deviation tracking | Reject sources >2 sigma from median  |
| services/l4/aggregation/ConflictResolver.ts | Handle divergent oracle responses             | Quorum-based selection (>=3 sources) |
| services/l4/config/oracle-sources.json      | Whitelist of oracle endpoints and weights     | Config-driven source management      |

### Hardcoded Consensus Rules (Safety Bounds)

- \* Minimum sources: 3
- \* Maximum deviation tolerance: 5% (price), 10% (sentiment)
- \* Weighted by oracle reputation score (from registry)

Module 4E: Schema Normalizer (Layer 3 Feed)

Responsibility: Transform raw aggregated data into the exact format Layer 3 (Intelligence) expects. This prevents schema drift and ensures type safety across the boundary.

File Inventory & Agent Tasks

| File                                          | Purpose                                      | Agent Task                     |
|-----------------------------------------------|----------------------------------------------|--------------------------------|
| services/l4/normalizer/NormalizationEngine.ts | Map all inputs to unified Layer3Input schema | Type transformers              |
| services/l4/schemas/layer3-input.schema.json  | JSON Schema validation                       | Strict schema enforcement      |
| services/l4/normalizer/Validators.ts          | Range checks, null checks, type guards       | Prevent garbage-in-garbage-out |

Normalized Output Schema (TypeScript)

```
interface Layer3Input {
  metadata: {
    batchId: string;           // UUIDv4
    timestamp: number;
    blockNumber: number;
    merkleRoot: string;
  };
  onChain: OnChainMetrics;
  offChain: {
    tokenPriceUsd: number;
    marketVolatility: number;
    sentimentScore?: number;
  };
  provenance: {
    signatures: string[];      // All oracle sigs
    sources: string[];         // Source identifiers
    consensusMethod: string;
  };
}
```

Module 4F: Tamper-Proof Buffer

Responsibility: Create a cryptographically secured commit-reveal buffer before Layer 3 consumes data. This is the critical circuit-breaker that prevents flash manipulation.

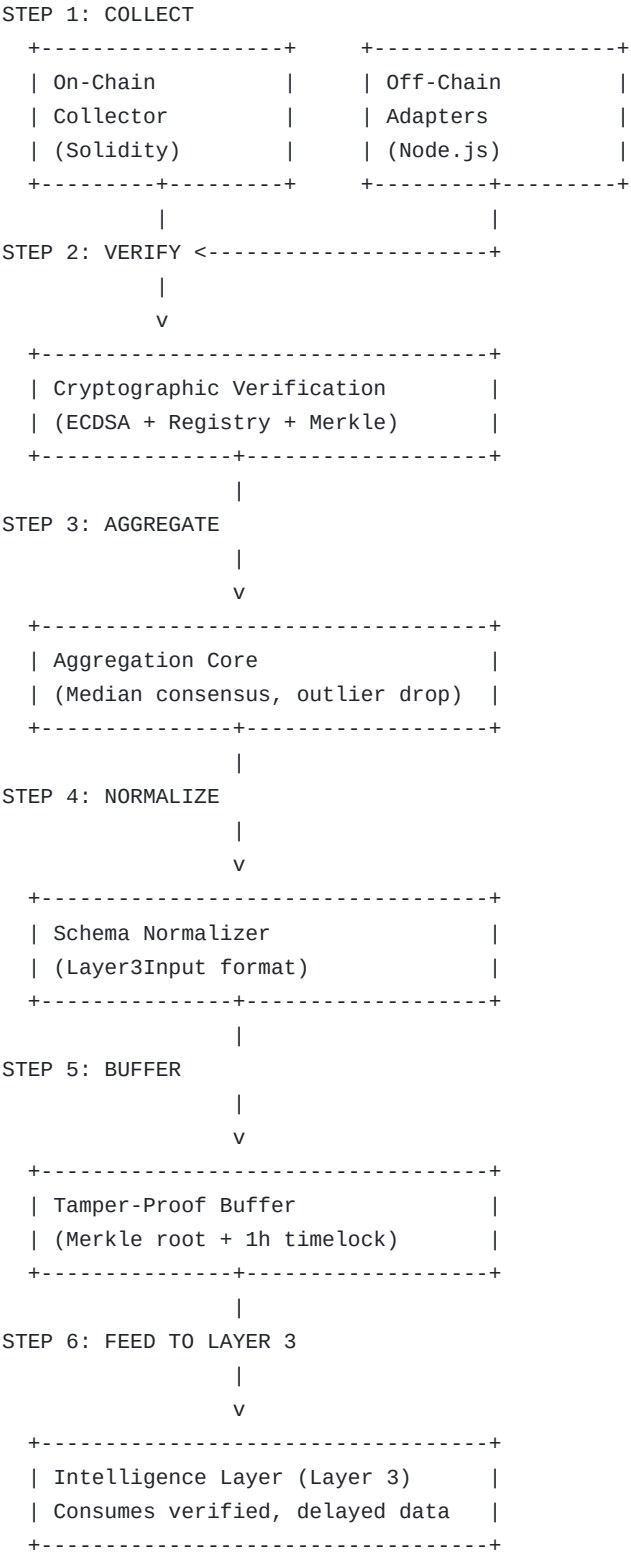
File Inventory & Agent Tasks

| File                                | Purpose                                               | Agent Task                               |
|-------------------------------------|-------------------------------------------------------|------------------------------------------|
| contracts/l4/DataBuffer.sol         | On-chain buffer storing Merkle roots with 1-hour lock | Commit Merkle root, enforce 1-hour delay |
| services/l4/buffer/CommitService.ts | Submit aggregated batch to on-chain buffer            | Web3 listener + submitter                |
| services/l4/buffer/AuditLogger.ts   | Immutable audit trail (IPFS/Arweave)                  | Log full batch + proofs                  |

Buffer Constraints

- \* Commit -> Reveal delay: 1 hour (prevents flash manipulation)
- \* Max batch size: 100 data points
- \* Automatic expiration: 24 hours if not consumed

### 3. Data Flow (Vertical Slice)



This six-step pipeline ensures that Layer 3 never acts on raw, unverified, or prematurely available data. Each step is independently testable and auditable.

## 4. Agentic IDE Implementation Prompts

---

The following prompts can be fed directly to your agentic IDE as independent, parallel workstreams. Each prompt is scoped to a single module boundary.

### Prompt A - Smart Contracts (Solidity)

Create the Layer 4 smart contract suite in `contracts/l4/`:

1. `OnChainDataProvider.sol` - view-only contract that reads DAO governance state (voter turnout, voice credits used, treasury flows) and returns a structured `OnChainMetrics` object.
2. `VerificationRegistry.sol` - admin contract storing approved oracle node addresses and their ECDSA public keys.
3. `DataVerifier.sol` - pure functions for on-chain Merkle proof verification.
4. `DataBuffer.sol` - commit-reveal buffer with 1-hour timelock, storing Merkle roots, emitting `DataCommitted(bytes32 batchId, bytes32 merkleRoot, uint256 revealTime)`.

All contracts must use Solidity ^0.8.19, OpenZeppelin Ownable2Step, and have comprehensive NatSpec. Write Foundry tests in `test/l4/`.

### Prompt B - Off-Chain Aggregation Service (TypeScript)

Build the off-chain aggregation service in `services/l4/`:

1. Implement adapter pattern for Chainlink price feeds and any supplementary APIs.
2. Build `ConsensusEngine.ts` that takes  $\geq 3$  oracle responses, calculates median, and rejects outliers beyond 2 standard deviations.
3. Build `MerkleBuilder.ts` that hashes normalized data packets and constructs a Merkle tree.
4. Create a cron-based `Aggregator.ts` that runs every 5 minutes, collects data, verifies signatures, reaches consensus, normalizes to `Layer3Input`, and submits the Merkle root to `DataBuffer.sol`.

Use `ethers.js v6`, include `.env.example`, and write Jest tests.



## Prompt C - Cryptographic Verification (TypeScript)

Implement the cryptographic verification layer in `services/l4/crypto/`:

1. `SignatureVerifier.ts` - Verify ECDSA signatures from oracle nodes against on-chain `VerificationRegistry`.
2. `MerkleBuilder.ts` - Build Merkle trees from `Layer3Input` arrays using `merkletreejs` and `keccak256`.
3. Ensure all verification functions are pure and return `{ valid: boolean; reason?: string }`.

Write unit tests with 100% branch coverage.

## Prompt D - Integration & E2E Testing

Create integration tests in `test/l4/integration/` that:

1. Spin up a local Anvil/Hardhat node with Layer 4 contracts deployed.
2. Mock 3 oracle nodes signing conflicting data.
3. Run the aggregation service end-to-end.
4. Verify the Merkle root lands in `DataBuffer` after consensus.
5. Test the timelock delay and Layer 3 consumption.

Use Docker Compose for service orchestration.

## 5. Critical Constraints Checklist

Before any agent marks a module "complete," the following constraints must be verified through both code review and automated testing.

| Constraint                          | Implementation Location                         | Test Requirement                             |
|-------------------------------------|-------------------------------------------------|----------------------------------------------|
| Cryptographically verifiable inputs | SignatureVerifier.ts + VerificationRegistry.sol | Reject unsigned data; reject wrong signer    |
| Multi-source aggregation            | ConsensusEngine.ts                              | Must require >=3 sources; drop outliers      |
| No direct Layer 3 execution         | DataBuffer.sol only emits data                  | Layer 3 cannot force treasury actions        |
| Tamper-proof audit trail            | AuditLogger.ts + Merkle roots                   | Full batch must be reconstructible from IPFS |
| Bounded manipulation risk           | 1-hour timelock in DataBuffer.sol               | Layer 5 veto window must exist               |

**WARNING:** Any module that violates these constraints must be vetoed by Layer 5.

## 6. Suggested File Tree

---

```

dao-governance/
|-- contracts/
|   |-- l4/
|   |   |-- OnChainDataProvider.sol
|   |   |-- VerificationRegistry.sol
|   |   |-- DataVerifier.sol
|   |   |-- DataBuffer.sol
|   |   |-- interfaces/
|   |       |-- IOnChainData.sol
|   |       |-- IVerificationRegistry.sol
|
|-- services/
|   |-- l4/
|   |   |-- adapters/
|   |       |-- BaseAdapter.ts
|   |       |-- ChainlinkMarketAdapter.ts
|   |       |-- SentimentAdapter.ts
|   |   |-- aggregation/
|   |       |-- Aggregator.ts
|   |       |-- ConsensusEngine.ts
|   |       |-- ConflictResolver.ts
|   |   |-- crypto/
|   |       |-- SignatureVerifier.ts
|   |       |-- MerkleBuilder.ts
|   |   |-- normalizer/
|   |       |-- NormalizationEngine.ts
|   |       |-- Validators.ts
|   |   |-- buffer/
|   |       |-- CommitService.ts
|   |       |-- AuditLogger.ts
|   |   |-- types/
|   |       |-- index.ts
|   |   |-- config/
|   |       |-- oracle-sources.json
|
|-- test/
|   |-- l4/
|   |   |-- OnChainDataProvider.t.sol
|   |   |-- DataBuffer.t.sol
|   |   |-- consensus.test.ts
|   |   |-- integration/
|   |       |-- full-pipeline.test.ts
|
|-- scripts/
|   |-- l4/
|   |   |-- collect-onchain.ts

```

This tree structure enforces clean separation between on-chain and off-chain logic, with each module owning its own directory. The agentic IDE can parallelize work across these boundaries without merge conflicts.

## 7. Appendix: Design Principles

---

### The Senses Principle

Layer 4 is explicitly named "The Senses" because, like biological sensory organs, it must filter noise, detect manipulation, and only transmit verified signals to the brain (Layer 3). A corrupted sensory layer causes the entire organism to misbehave.

### Why Commit-Reveal?

The 1-hour timelock between commit and reveal serves two purposes: (1) it prevents oracle nodes from frontrunning parameter adjustments, and (2) it provides Layer 5 (the Failsafe/Veto Layer) a fixed window to inspect and potentially cancel malicious data feeds before Layer 3 acts on them.

### Why Merkle Trees?

Merkle trees allow Layer 3 to verify the integrity of a large batch of data by checking a single 32-byte root. This is gas-efficient on-chain while preserving the ability to prove inclusion of any individual data point. The audit logger stores the full tree off-chain (IPFS/Arweave) for transparency.

### Agentic Parallelization Strategy

The six modules (4A-4F) are designed to be implemented in parallel by separate agent instances because they communicate only through well-defined interfaces. Module 4A and 4B can be built simultaneously. Module 4C depends only on the interface definitions of 4A and 4B. Module 4D depends on 4C. Module 4E depends on 4D. Module 4F depends on 4E. This creates a natural dependency DAG that the IDE can schedule automatically.